Design a Ticketing System (BookMyShow)

Mastering System Design: From Basics to Cracking Interviews



1 User
Search & browse movies

2 Browse Movies & Shows
Select movie, theatre, date and show time

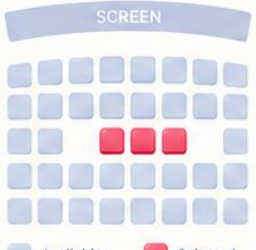


Inception
Action | Sci-Fi
★ 8.8 (500K reviews)

Choose Show Time

10:00 AM | 01:00 PM | 07:00 PM

3 Select Seats
Choose your preferred seats



4 Temporary Seat Lock
Seats are temporarily reserved for a few minutes



04:32

Seat lock prevents double booking

5 Payment
Complete the payment to confirm booking

- Credit / Debit Card
- UPI
- Net Banking
- Wallet

Pay Now

6 Confirmed Ticket
Booking successful! Enjoy the movie!



Inception
PVR: Phoenix Mall
Sat, 12 Oct 2024
01:00 PM

Audi 1
Seats: B10, B11, B12

Booking Confirmed

System Architecture (High Level)



Users
Web / Mobile App

API Gateway
Routing, Auth, Rate Limiting

Booking Service
Handles search, booking & seat lock

Seat Inventory Service
Manages seat availability and locks (distributed)

Payment Service
Processes payments (success / failure)

Notification Service
Sends booking confirmation (Email/SMS)

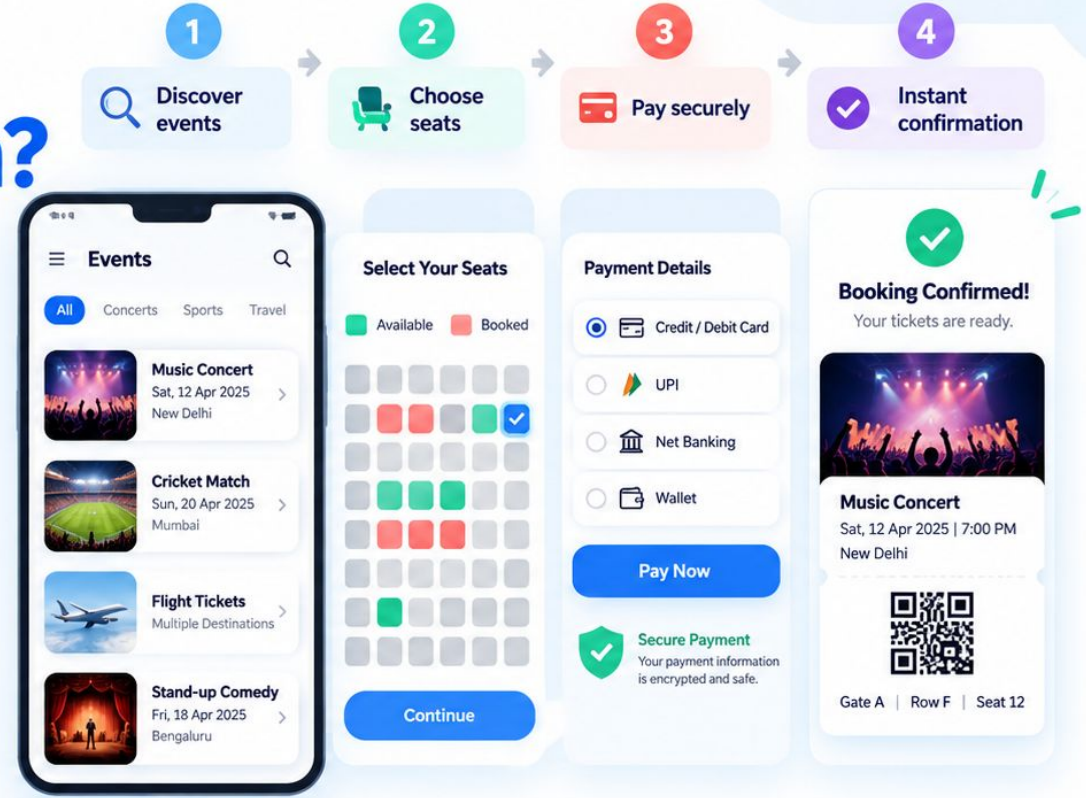
Database
Stores users, bookings, payments, shows, etc.

Instructor: Rahul Rajat Singh

What is a Ticketing System?

A ticketing system is an online platform that allows users to **browse, book, and manage tickets** for events such as concerts, sports, and travel.

It must handle **real-time seat availability**, **secure transactions**, and **high concurrency**.



Functional Requirements

User Functions

Users should be able to:

1. Browse events and view available seats
2. Book/reserve tickets
3. Get real-time seat availability
4. Pay for tickets securely
5. Receive email/SMS confirmations

Admin Functions

Admins should be able to:

1. Create/manage events and venues
2. Define seat layouts and pricing



Non-Functional Requirements

Seat Booking System



High Availability

No downtime during peak sales.



Low Latency

Booking response in milliseconds.



Scalability

Handle flash sales and global traffic.



Data Consistency

Avoid double booking of the same seat.



Audit Logs

For tracking transactions and fraud prevention.

Constraints & Challenges

5M total users
100K concurrent users at peak

Global event organizers
Multi-region support

Handling payment failures
Release locked seats quickly



Estimating Scale & Identifying Bottlenecks

User Load Assumptions

1M
Daily Active Users (DAU)

100K
Concurrent Users during peak events

~10
Events browsed per user/day

10M **Read requests/day**



Booking Traffic Estimations

~500K
Bookings/day

Average: ~6
Bookings/sec

Peak: up to 2,000
Bookings/sec

e.g., Big concert ticket release



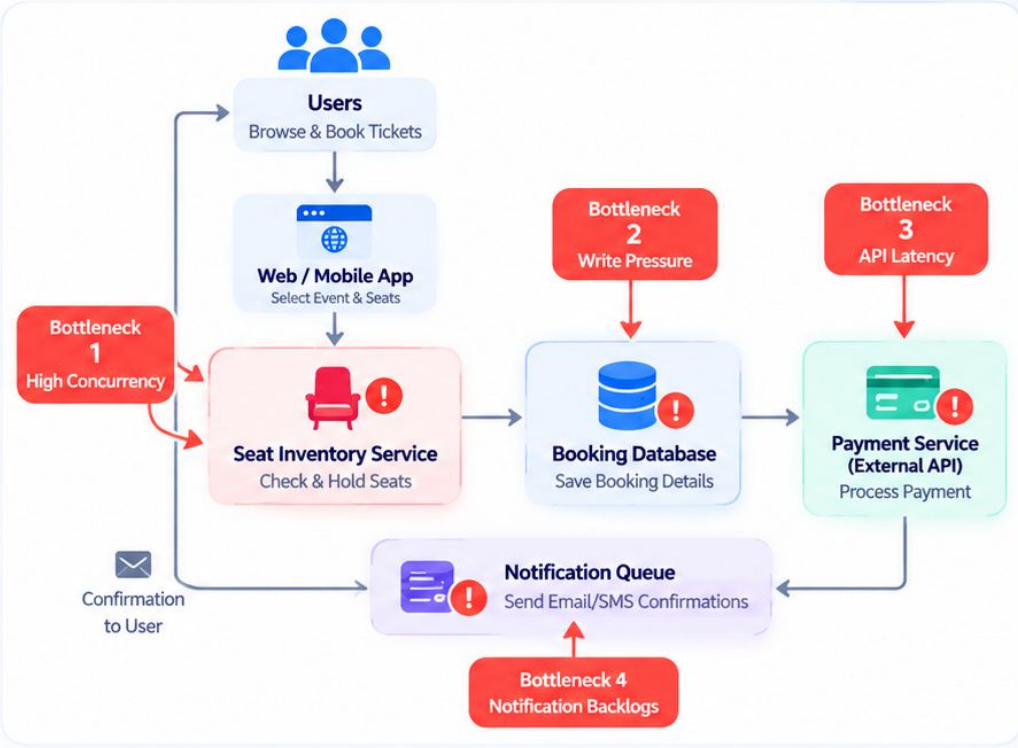
Identifying System Bottlenecks

1. **Concurrency in Seat Allocation**
   Race conditions when multiple users book the same seat.

2. **Database Write Pressure**
   Booking spikes can overwhelm the write database.

3. **Payment & External API Latency**
   Third-party delays or failures can block seat availability.

4. **Notification Backlogs**
   Email/SMS confirmations can queue up during traffic spikes.



Core Components Overview

Major components

The system consists of the following major components:

Frontend Clients
Web & Mobile App

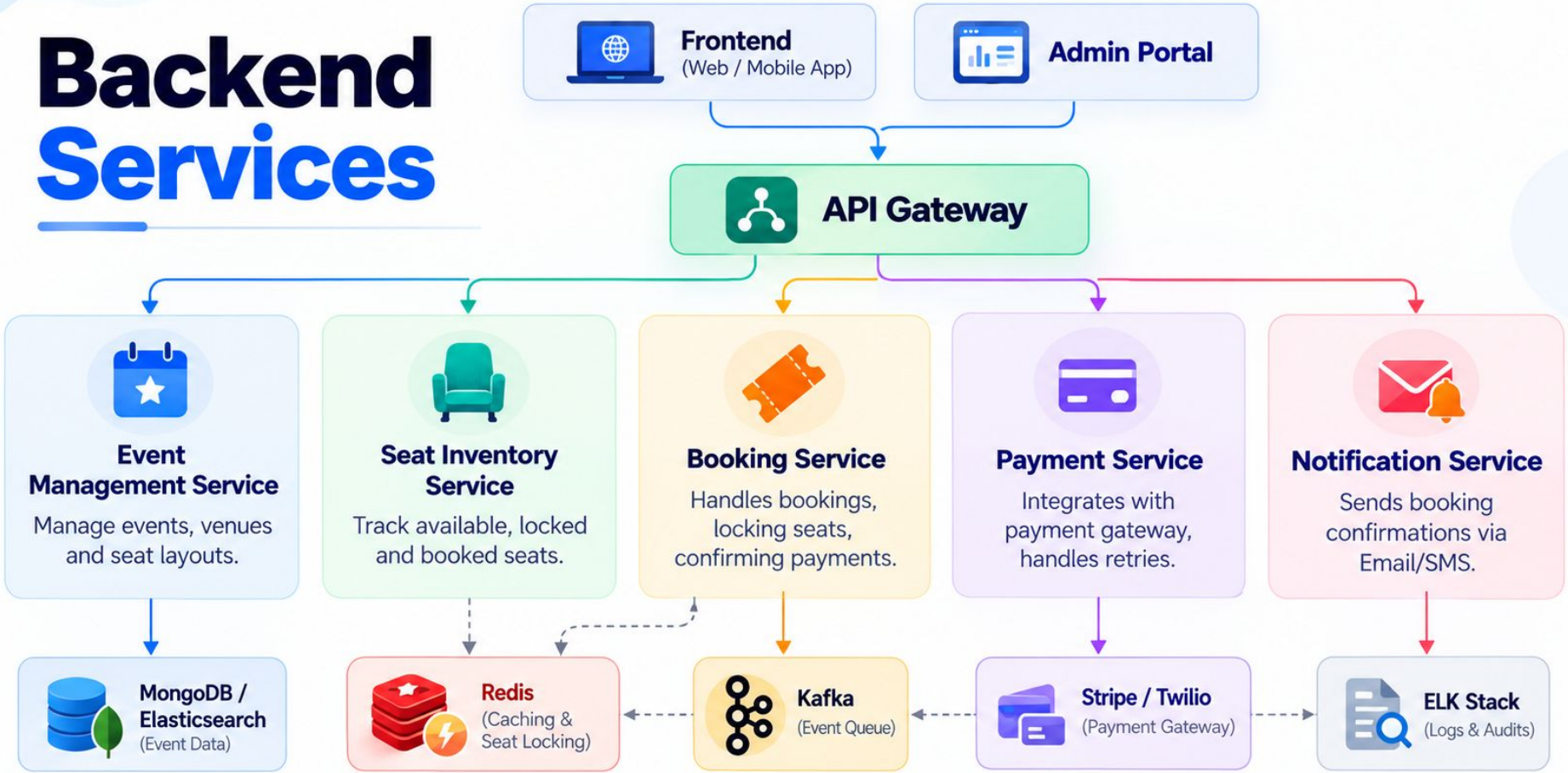
API Gateway
Unified entry point for routing and authentication

Authentication Service

Admin Portal
Event creation, venue setup, pricing



Backend Services



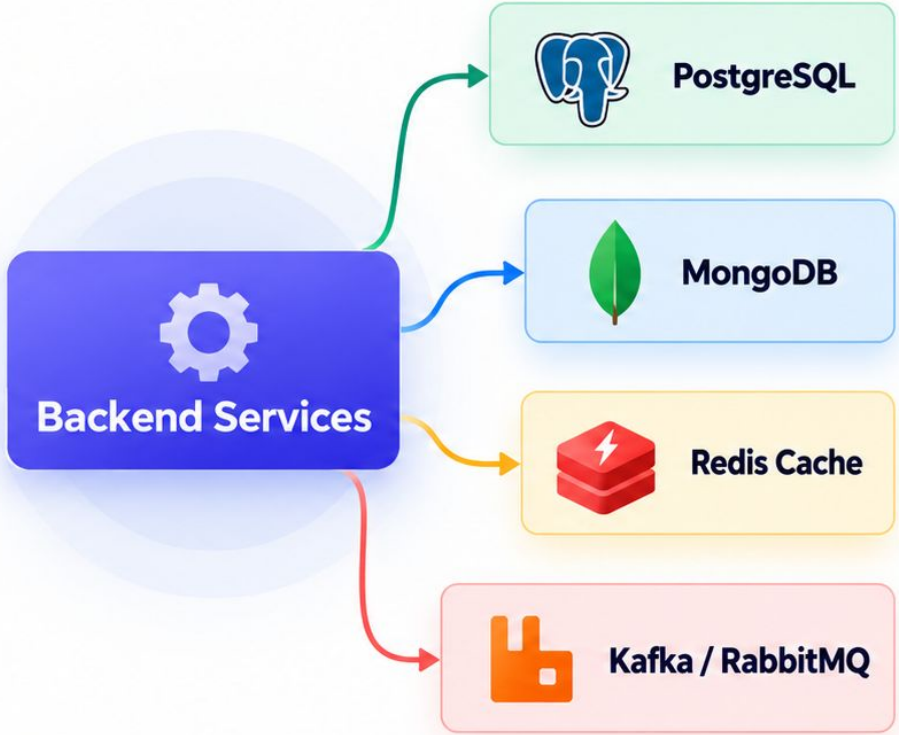
Data & Caching Architecture

1. **Relational DB (PostgreSQL/MySQL)**
   For transactions, bookings, users.

2. **NoSQL DB (MongoDB/DocumentDB)**
   For events and seat layouts.

3. **Caching Layer (Redis/Memcached)**
   Real-time seat availability.

4. **Queue System (Kafka/RabbitMQ)**
   Async handling for:
   - Emails
   - Payment retries
   - Audit logs



Notable Design Decisions



1 Concurrency Control

Use **optimistic locking** (version checks) or **pessimistic seat-level locks.**



2 Seat Hold Timeout

Redis-backed TTL lock automatically releases seats after **5 minutes**.



3 CQRS Pattern

Split **reads** (seat availability, listings) from **writes** (bookings).



4 Payment Idempotency

Idempotency keys prevent duplicate charges and enable safe retries.

API Design — Key Endpoints

Event Management Service

Base Resource: `/events`

Manages events like concerts, sports, and travel with details such as name, description, date, time and location.

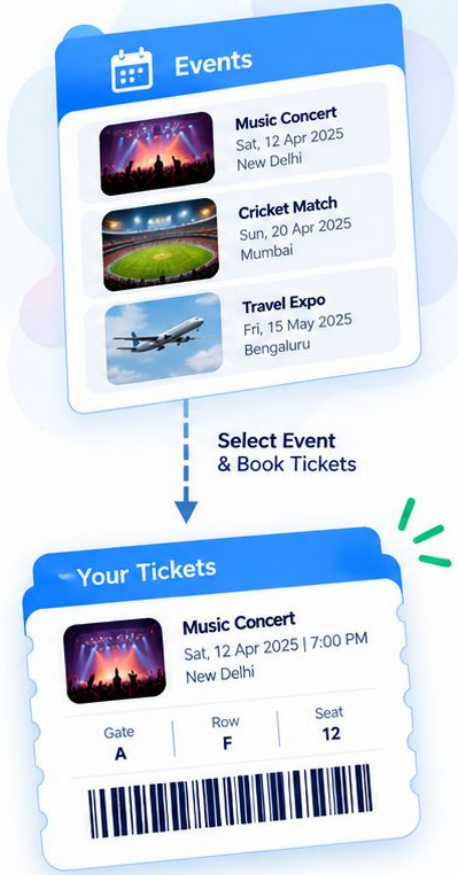
POST	`/events`	**Create a new event** Create a new event with event details.	Request (JSON) **name**, description, date, time, location, ...	Response **201 Created** (with event ID)
GET	`/events`	**Get all events** Supports filtering, sorting and pagination via query parameters.	Query Parameters filter, sort, page, pageSize, ...	**200 OK** (array of events)
GET	`/events/{eventId}`	**Get event details** Get details of a specific event.	Path Parameter **eventId**	**200 OK** (event details) **404 Not Found** (event not found)

Booking Service

Base Resource: `/bookings`

Handles ticket bookings, manages seat availability and creates bookings for events.

POST	`/bookings`	**Create a new booking** Create a new booking for an event.	Request (JSON) eventId, userId, numberOfTickets, ...	**201 Created** (with booking ID) **400 Bad Request** (e.g., not enough tickets)
GET	`/bookings`	**Get all bookings** Supports filtering by user or event via query parameters.	Query Parameters userId, eventId, page, pageSize, ...	**200 OK** (array of bookings)
GET	`/bookings/{bookingId}`	**Get booking details** Get details of a specific booking.	Path Parameter **bookingId**	**200 OK** (booking details) **404 Not Found** (booking not found)



Strategic Tech & Infra Decisions

1. API & Security

Secure entry, routing and user authentication

1 API Gateway

Use NGINX for self-managed deployments or AWS API Gateway for serverless routing and rate limiting.

NGINX | AWS API Gateway

2 Authentication

Implement OAuth 2.0 with JWT tokens for secure, stateless authentication.

OAuth 2.0 | JWT

2. Data & Performance

Reliable storage, fast access and flexible search

3 Booking Database

Choose PostgreSQL for strong consistency and transactional support.

PostgreSQL

4 Event & Venue Data

Use MongoDB or Elasticsearch for flexible, schema-less search and filtering.

MongoDB | Elasticsearch

3. Integrations & Messaging

Payments, notifications and asynchronous processing

5 Caching Layer

Integrate Redis for fast access to seat availability and temporary locks.

Redis

6 Async Messaging

Adopt Kafka for high-throughput event streaming and async workflows (notifications, logging).

Apache Kafka

7 Payment Gateway

Integrate with Stripe or Razorpay, ensuring support for retries and webhooks.

stripe | Razorpay

8 Notifications

Use AWS SES for emails and Twilio for SMS confirmations.

aws SES | Twilio

4. Infrastructure & Observability

Scalable, resilient and observable systems

9 Infrastructure

Deploy on Kubernetes with Auto-Scaling Groups for elasticity and resilience.

Kubernetes | Auto-Scaling Groups

10 Monitoring & Logging

Leverage Prometheus and Grafana for metrics and real-time dashboards. Use the ELK Stack (Elasticsearch, Logstash, Kibana) for centralized log aggregation and search.

Prometheus | Grafana | ELK Stack (Elasticsearch, Logstash, Kibana)

The Final Design - Ticketing System (BookMyShow)

